

## **Seminar 3: PROGRAMMING CONDITIONAL STRUCTURES**

### **ALTERNATIVE STRUCTURES**

#### ***IF..THEN..END IF structure***

```
IF cond1 THEN
    secvcom1
ELSE
    secvcom2
END IF;
```

```
IF cond1 THEN
    secvcom1
ELSE
    IF cond2 THEN
        secvcom2
    END IF;
END IF;
```

- ELSIF may be used instead of nested IF branches

```
IF cond1 THEN
    secvcom1
ELSIF cond2 THEN secvcom2;
ELSIF cond3 THEN secvcom3;
-----
ELSIF condN THEN secvcomN;
ELSE secvcomN+1;
END IF;
```

#### **Example:**

Based on the price list of a product having the id read from the keyboard, display the modified price in the console.

```
SET SERVEROUTPUT ON
DECLARE
v_listPrice product_information.list_price%type;
BEGIN
SELECT list_price into v_listPrice from product_information where product_id=&p;
dbms_output.put_line ('The initial price is : '||v_listPrice);
IF v_listPrice < 500 THEN
    v_listPrice:=2* v_listPrice;
ELSIF v_listPrice between 500 and 1000 THEN
    v_listPrice:=1.5 * v_listPrice;
ELSE
    v_listPrice:=1.25* v_listPrice;
END IF;
dbms_output.put_line('The final price is: '||v_listPrice);
end;/
```

**Pay attention to the NULL type variables and condition evaluation!**

For example, the following example will display „Congrats!” because the *grade* variable is declared but not initialized:

```
DECLARE
grade number;
Begin
IF grade<5 THEN
dbms_output.put_line(Sorry');
ELSE
dbms_output.put_line(Congrats');
END IF;
END;
/
```

Test the scenarios defined below:

```
X:=10;
Y:=NULL;
IF x!=y then
--returns NULL and not TRUE
END IF;
```

or

```
a:=NULL;
b:=NULL;
IF a=b then
--returns NULL and not TRUE
END IF;
```

***CASE ... WHEN... THEN... structure***

There are 2 options:

- ***CASE Expressions*** returning a result into a variable. It ends in END.
- ***CASE Statement*** executing a specific instruction. It ends with END CASE and each row has „;”

***CASE Expressions:***

Variable:=

```
CASE [Selector]
  WHEN expression1 THEN result1
  WHEN expression2 THEN result2
  -----
  WHEN expressionN THEN resultN
  [ELSE result N+1]
END;
```

**Example:**

```
SET SERVEROUTPUT ON
DECLARE
v_listPrice product_information.list_price%type;
BEGIN
SELECT list_price into v_listPrice from product_information where product_id=&p;
dbms_output.put_line ('The price list is: '||v_listPrice);

v_listPrice:= CASE WHEN v_listPrice < 500 THEN 2* v_listPrice
                  WHEN v_listPrice between 500 and 1000 THEN 1.5 * v_listPrice
                  ELSE 1.25* v_listPrice END;

dbms_output.put_line('The final price is: '||v_listPrice);
end;
/
```

**CASE Statement:**

```
CASE [Selector]
  WHEN expression1 THEN action1;
  WHEN expression2 THEN action2;
  -----
  WHEN expressionN THEN actionN;
  [ELSE action N+1];
END CASE;
```

**Conditional structure with CASE without a selector:**

```
SET SERVEROUTPUT ON
DECLARE
v_listPrice product_information.list_price%type;
BEGIN
SELECT list_price into v_listPrice from product_information where product_id=&p;
dbms_output.put_line ('Pretul de lista initial este: '||v_listPrice);
CASE
WHEN v_listPrice < 500 THEN
  v_listPrice:=2* v_listPrice;
WHEN v_listPrice between 500 and 1000 THEN
  v_listPrice:=1.5 * v_listPrice;
ELSE
  v_listPrice:=1.25* v_listPrice;
END CASE;
dbms_output.put_line('Pretul final este: '||v_listPrice);
end;
/
```

**Warning:** The lack of the ELSE clause in a CASE STATEMENT can lead to an exception if none of the WHEN expressions evaluate to TRUE. This does not occur in the case of IF or CASE EXPRESSION.

## Repetitive structures

### *LOOP.....END LOOP structure*

LOOP

```
    commands;  
    EXIT [WHEN cond];  
END LOOP;
```

#### **Example:**

Display using a loop...end loop structure the numbers 9,7,4,0.

```
set serveroutput on  
DECLARE  
v_nr number(2):=10;  
i number(2):=1;  
BEGIN  
loop  
v_nr:=v_nr-i;  
i:=i+1;  
exit when v_nr < 0;  
dbms_output.put_line(v_nr);  
end loop;  
END;  
/
```

Display in ascending order the employees with the id between 100-110 if their salary is lower than the average:

```
DECLARE  
v_sal employees.salary%type;  
v_avgSal v_sal%type;  
i number(4):=100;  
BEGIN  
SELECT avg(salary) into v_avgSal from employees;  
dbms_output.put_line('The average salary is: '||v_avgSal);  
loop  
select salary into v_sal from employees where employee_id=i;  
dbms_output.put_line('The employee with id '||i||' has the salary: '||v_sal);  
i:=i+1;  
exit when v_sal<v_avgSal or i>110;  
end loop;  
end;  
/
```

### ***WHILE.....LOOP.....END LOOP structure***

**WHILE** cond **LOOP**

    Command sequence 1;

    Command sequence 2;

    EXIT [WHEN cond];

**END LOOP**;

#### **Exemplu:**

Display using a loop...end loop structure the numbers 9,7,4,0.

```
set serveroutput on
DECLARE
v_nr number(2):=10;
i number(2):=1;
BEGIN
while v_nr > 0
loop
v_nr:=v_nr-i;
i:=i+1;
dbms_output.put_line(v_nr);
end loop;
END;
/
```

Display in ascending order the employees with the id between 100-100 if their salary is lower than the average:

```
DECLARE
v_sal employees.salary%type;
v_avgSal v_sal%type;
i number(4):=100;
BEGIN
SELECT avg(salary) into v_avgSal from employees;
dbms_output.put_line('The average salary is: '||v_avgSal);
while i<=110 loop
select salary into v_sal from employees where employee_id=i;
dbms_output.put_line('The employee with the id '||i||' has the salary: '||v_sal);
i:=i+1;
exit when v_sal < v_avgSal;
end loop;
end;
/
```

### ***FOR.....LOOP....END LOOP structure***

```
FOR var IN [REVERSE] valmin..valmax LOOP
    Secventa comenzi;
    EXIT [WHEN cond];
END LOOP;
```

#### **Obs:**

The range values can be of any type, but they must have values that can be converted to an integer (e.g., 20/13 or 11/5). If these two values are equal as integers, then the instructions inside the loop will execute only once. For example, the sequence:

```
FOR i IN 3..3 LOOP
    Commands sequence;
END LOOP;
```

#### **Example:**

Display using a for...loop...end loop structure the numbers 9,7,4,0.

```
set serveroutput on
DECLARE
v_nr number(2):=10;
i number(2);
BEGIN
for i in 1..10
loop
v_nr:=v_nr-i;
exit when v_nr < 0;
dbms_output.put_line(v_nr);
end loop;
END;
/
```

Employees with codes in the range 100-110 are displayed in ascending order as long as their salary is lower than the average.

```
DECLARE
v_sal employees.salary%type;
v_avgSal v_sal%type;
-- i does not have to be declared
BEGIN
SELECT avg(salary) into v_avgSal from employees;
dbms_output.put_line('The average salary is: '||v_avgSal);
for i in 100..110 loop
select salary into v_sal from employees where employee_id=i;
dbms_output.put_line('The employee with the id '||i||' has the salary: '||v_sal);
exit when v_sal<v_avgSal;
end loop;
end;
/
```

**Display the number of orders for each employee whose ID is in the range 155 to 160, but stop displaying as soon as the first employee in this range with no orders is found.**

**Load in the MESSAGES table numbers from 1...50 excepting the numbers 16 și 7.**

```
CREATE TABLE MESSAGES (REZ VARCHAR2(30));  
  
BEGIN  
FOR i IN 1..10 LOOP  
    IF i=16 OR i=7 THEN  
        NULL;  
    ELSE  
        INSERT INTO MESSAGES(REZ) VALUES (i);  
    END IF;  
    COMMIT;  
END LOOP;  
END;  
/
```

***LOOP nested structures (loop labeling)***

```
BEGIN  
    <<OUTER_LOOP >>  
    LOOP  
        v_var:=v_var+1;  
        EXIT WHEN v_var>10;  
        <<INNER_LOOP>>  
        LOOP  
            .....  
            EXIT OUTER_LOOP WHEN cond1;  
            EXIT WHEN cond2;  
            .....  
        END LOOP;  
        .....  
    END LOOP;  
    .....  
END LOOP;  
END;
```